INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD

DS2020 INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Lab 3

Students:
Abhishek Iyer (112401002)
Kalakuntla Parjanya (112401018)

Instructor:
Dr. Narayanan C Krishnan

March 13, 2026

INDIAN INSTITUTE
OF TECHNOLOGY
PALAKKAD

Question 1

Describe the heuristic that was used, along with the rationale behind using it.

Answer: The heuristic evaluates the board in two parts: controlling the center and finding 4-in-a-row threats using a "sliding window."

First, flat bonus points are added to pieces playing in the center of the board, because the middle columns are statistically more involved in winning lines than the edges.

Second, the code automatically slices the board into every possible 1x4 window (horizontal, vertical, and diagonal). If a window has pieces of both the players, it isn't considered. If it only has pieces from one player, the algorithm counts how many pieces they have in the window and scores them using these tables:

Tiles	Air Score	Grounded Score
0	0	0
1	0.5	1
2	3	10
3	10	100

Table 1: MAX Player (My) Weights

Tiles	Air Score	Grounded Score
0	0	0
1	0.5	1
2	3	4
3	12	500

Table 2: MIN Player (Opponent) Weights

Rationale

This heuristic is built focusing on a few key concepts:

- **Gravity Checking:** This heuristic uses a gravity check that looks in the board array to see if the empty spot in a 3-in-a-row is actually playable right now (AKA "grounded"). If it is grounded, the score gets increased massively, because it's a real threat on the very next turn.
- **Prioritize Defensive:** The opponent's grounded 3-in-a-row is scored at 500, which is much higher than the MAX player's own 3-in-a-row (100). This forces the AI to prioritize blocking an immediate loss over trying to greedily build its own win.
- **Building Fork through Weights:** The AI values its own grounded 2-in-a-rows at 10 points, but the opponent only values blocking them at 4 points. This slight imbalance is intentional - it teaches the AI to aggressively stack double-threats on the bottom of the board, which the opponent will theoretically ignore until it degenerates into an unblockable fork!
- **Speed vs Depth:** Using NumPy array slicing instead of regular nested Python lists makes everything run incredibly fast. Because the math is cheap to compute, the AI can comfortably search to Depth 5 within the 5-second limit without timing out.

Question 2

Given different time constraints, how will your algorithm perform? How much of the tree can you explore given 3, 5, or 10 seconds per turn?

Answer: Alpha-Beta Pruning Algorithm

Using the `--time` parameter, we tested the algorithm for various time limits with varying depths to identify the highest depth achievable without a time-out. The results are summarized below:

- **3s:** Depth 5
- **5s:** Depth 6
- **10s:** Depth 6

As depth increases, the execution time grows exponentially. This is because each additional level of depth explores b more states (where b is the branching factor). While Alpha-Beta pruning significantly reduces the search space by eliminating suboptimal branches, the complexity remains exponential.

Depth	Maximum Time (seconds)
3	0.1249
4	0.3423
5	2.6021
6	3.5372
7	33.4538
8	42.4144

Table 3: Maximum execution time per move (Alpha-Beta)

The data suggests that for standard computation limits (3 seconds), a depth of 5 is the most reliable configuration. The efficiency of pruning allows the agent to look further ahead than a standard minimax search, but the exponential growth of the game tree becomes too large to handle beyond depth 7, where the search time exceeds 30 seconds.

Expectimax Algorithm

A similar analysis was performed for the Expectimax algorithm. The highest searchable depths within the time constraints are:

- **3s:** Depth 4
- **5s:** Depth 5
- **10s:** Depth 5

The time required for Expectimax grows significantly faster than Alpha-Beta. This performance variation is due to the following factors:

1. **Lack of Pruning:** Unlike Alpha-Beta, which prunes branches that cannot influence the final decision, Expectimax is an exhaustive search. To calculate a mathematically accurate *expected value*, it must explore every possible node at a given depth.
2. **Chance Node Complexity:** This algorithm treats opponent moves as chance nodes where each valid column has an equal probability of selection ($P = 1/n$). Expectimax must evaluate the utility of every valid move to compute a weighted average, whereas Alpha-Beta only seeks the best/worst case.
3. **Exponential Expansion:** Without pruning, the number of nodes evaluated is strictly $O(b^d)$. This explains the sharp jump between Depth 5 (3s) and Depth 6 (20s), as the algorithm can no longer rely on early exits from the search tree.

Depth	Maximum Time (seconds)
3	0.0690
4	0.4362
5	3.0428
6	21.7508
7	157.8944

Table 4: Maximum execution time per move (Expectimax)

Because Expectimax must account for the probability of every valid move, its computational times are significantly higher than Alpha-Beta. While it performs well against a Random Player by calculating expected utility, its depth is effectively capped at 5 to avoid timing out during the 5-second window. Searching at depth 6 or higher is unfeasible for real-time play due to the exhaustive nature of the chance-node calculations.
A graph showcasing the difference in maxmum time to compute a move is given below:

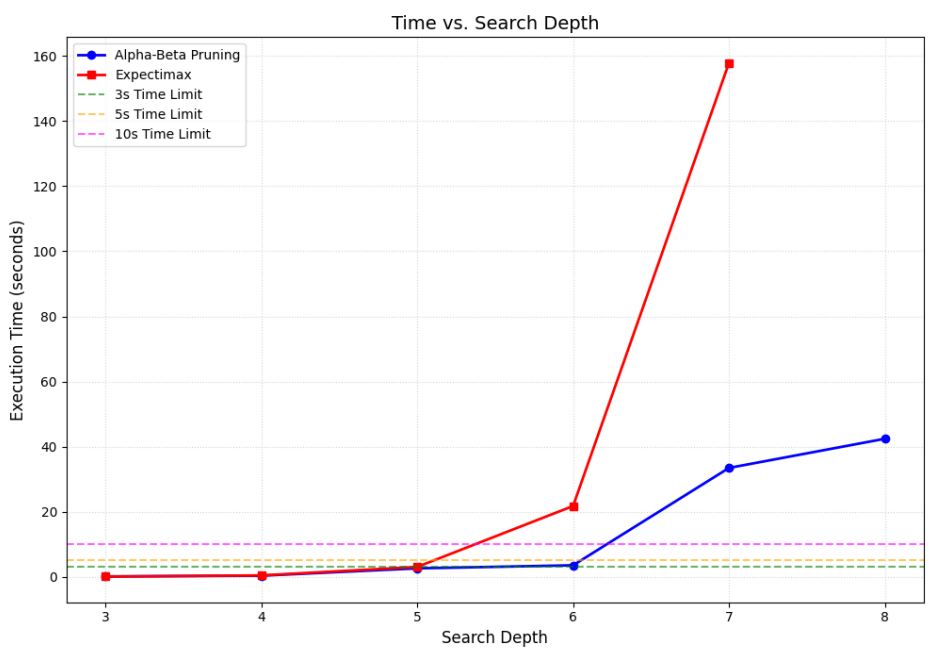


Figure 1: Time vs Search Depth

Question 3

If the AI player plays against itself, does the player who goes first do better or worse in general? Illustrate it through some results.

Answer: Generally, Player 1 has a massive advantage because they get to take the middle column first. Since the center column is part of the most possible winning combinations, whoever controls it usually controls the game. When our AI plays against itself, we found that Player 1 wins almost every time, provided the search depth is high enough to actually "see" a long-term plan.

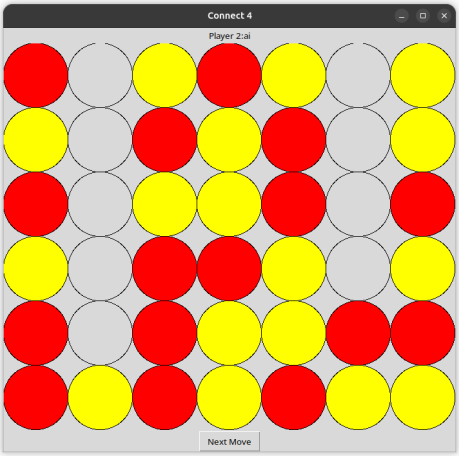
The table below shows who won during our play tests at different depths:

Max Depth	Winner
3	Player 2
4	Draw / Tie
5	Player 1
6	Player 1
7	Player 1
8	Player 1

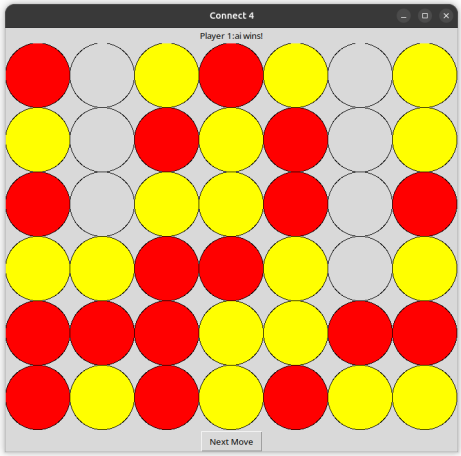
Table 5: Play test results at different depth limits.

Analysis of Results

1. Depth 5 and above: Player 1 Dominance
Once we set the depth to 5 or more, Player 1 dominates. Because our evaluation function gives a lot of points for 3-in-a-rows, Player 1 eventually sets up a "fork" - two separate winning lines that can't both be blocked in one turn. This forces Player 2 into a state of **Zugzwang**, where any legal move they make only helps Player 1 complete one of their threats. This is why Player 1 wins every game from depth 5 up to depth 8.



(a) Depth 6: P1 stacking the center to set up traps.

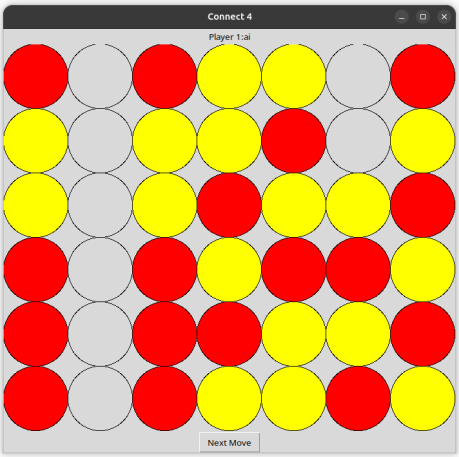


(b) Depth 6: P1 wins.

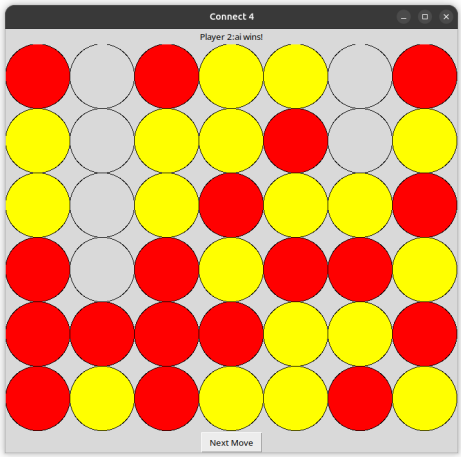
Figure 2: Player 1 using deeper search to force a win.

2. Depths 3 and 4: Shallow Search Errors

At low depths, the AI is pretty ”shortsighted.” At depth 4, both players basically just react to each other until the board fills up, usually resulting in a tie. Interestingly, at depth 3, Player 2 actually wins. This is likely due to the ”Horizon Effect.” Since Player 1 can only see 3 moves ahead, it might make a move that looks good in the short term but is actually a trap. Because Player 2 moves *after* Player 1, its 3-move search effectively sees one step further into the game’s future, allowing it to spot and exploit Player 1’s mistake.



(a) Depth 3: P2 can win no matter what P1 plays.



(b) Depth 3: P2 wins.

Figure 3: Player 2 exploiting shallow search mistakes.